

ARAMA ALGORİTMALARI

Linear · Binary · Jump · Interpolation/Dictionary · Quadratic Binary Search
Kod + Analiz + Θ İspatı

Temel Sorular: (1) Hangi koşulda çalışır? (2) İterasyon sayısı nedir? (3) Zaman karmaşıklığı Θ nedir? (4) Rakip algoritmalarla kıyasla.

1. Linear Search – Sırasız Verilerde

Diziyi baştan sona teker teker tarar. Ek koşul gerekmez; sıralı olmayan her veri yapısında uygulanabilir.

Linear Search – Sırasız Verilerde Arama

Kod

```
int linearSearch(int arr[], int n, int item) {
    for (int i = 0; i < n; i++)
        if (arr[i] == item) return i;
    return -1;
}
```

Olasılıklı Analiz

P_i = i 'inci pozisyondaki elemanın aranma olasılığı; P_n = elemanın dizide olmaması olasılığı

$$\sum_{i=0}^{n-1} P_i + P_n = 1 \implies P = \frac{1 - P_n}{n} \text{ (uniform)}$$

$$T(n) = n \cdot P_n + \sum_{i=0}^{n-1} (i+1) P_i = n \cdot P_n + \frac{n(n+1)}{2} \cdot \frac{1 - P_n}{n} = n \cdot P_n + \frac{(n+1)(1 - P_n)}{2}$$

$$T(n) = \frac{n \cdot P_n \cdot 2 + (n+1) - (n+1)P_n}{2} = \frac{n+1}{2} + P_n \cdot \frac{n-1}{2}$$

Özel durumlar: $P_n = 0$ (her zaman bulunur): $T(n) = \frac{n+1}{2}$ | $P_n = 1$ (hiç bulunmaz): $T(n) = n$

Θ ispatı: $f(n) = n$, $0 \leq P_n \leq 1$ olduğundan

$$\frac{n+1}{2} \leq T(n) \leq n \implies c_1 n \leq \frac{n+1}{2} \leq T(n) \leq n \leq c_2 n$$

$n_0 = 1, c_1 = c_2 = 1$ seçilirse:

$$T(n) = \Theta(n)$$

Pratik – Linear Search

(a) $P_n = 0.5$ (eleman %50 olasılıkla dizide) iken $T(n)$ formülünü hesaplayın. _____

(b) Dizide hiç tekrar yoksa en iyi durum kaç adımdır? _____

2. Binary Search – Sıralı Verilerde

Her adımda arama aralığını yarıya bölür. $O(\log n)$ adımda bulur. Dizinin **sıralı** olması şarttır.

Binary Search – Sıralı Verilerde Arama

İteratif Kod

```
int bs_iterative(int arr[], int n, int target) {
    int left = 0, right = n-1;
    while (left <= right) {
        int mid = left + (right - left)/2;
        if (arr[mid] == target) return mid;
        else if (arr[mid] < target) left = mid + 1;
        else right = mid - 1;
    }
    return -1;
}
```

Rekürsif Kod

```
int bs_recursive(int arr[], int left, int right, int target) {
    if (left > right) return -1;
    int mid = left + (right - left)/2;
    if (arr[mid] == target) return mid;
    else if (arr[mid] < target) return bs_recursive(arr, mid+1, right, target);
    else return bs_recursive(arr, left, mid-1, target);
}
```

İterasyon / Yineleme Analizi

Her adımda $n \rightarrow n/2 \rightarrow n/4 \rightarrow \dots \rightarrow 1$: toplam k adım, $n/2^k = 1 \Rightarrow k = \log n$

Yineleme bağıntısı:

$$T(n) = T(n/2) + c, \quad T(1) = c$$

$$T(n) = T(n/2^i) + ic \xrightarrow{i=\log n} T(n) = T(1) + c \log n = c(\log n + 1)$$

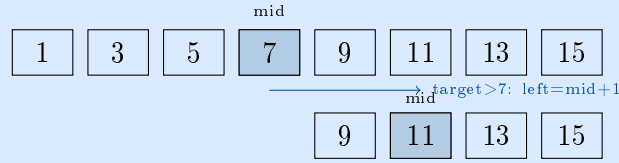
Θ ispatı: $f(n) = \log n$

$$c_1 \log n \leq c(\log n + 1) \leq c_2 \log n \implies c_1 \leq c + \frac{c}{\log n} \leq c_2$$

$n_0 = 2$, $c_1 = c_2 = 3$ seçilirse ($c = 1$ alınırsa $1 + \frac{1}{\log 2} = 2 \leq 3$):

$$T(n) = \Theta(\log n)$$

Görsel: Aralık Yarılanması



Pratik – Binary Search

(a) $\{2, 5, 8, 12, 16, 23, 38, 56, 72, 91\}$ dizisinde $\text{target}=23$ 'ü ara. Her adımda left , right , mid değerlerini yazın. _____

(b) $n = 10^6$ için Binary Search en fazla kaç adım atar? ($\log_2 10^6 \approx 20$) _____

3. Jump Search – Sıralı Verilerde Blok Atlama

Blok blok ilerler, hedef aşılmıca blok içinde doğrusal arama yapar. Blok boyutu $j = \sqrt{n}$ seçilince $\Theta(\sqrt{n})$ elde edilir.

Jump Search – Sıralı Verilerde Blok Atlama

Kod

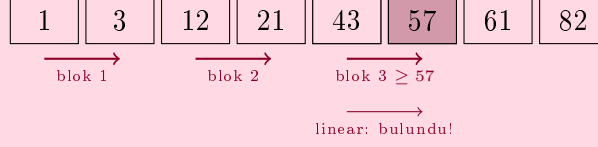
```
int jumpSearch(int arr[], int n, int item) {
    int j = sqrt(n);
    int lowerBound = 0, upperBound = j;
    while (arr[upperBound] <= item && upperBound < n) {
        lowerBound = upperBound;
        upperBound += j;
        if (upperBound > n-1) { upperBound = n; break; }
    }
    for (int i = lowerBound; i <= upperBound; i++)
        if (arr[i] == item) return i;
}
```

```

    return -1;
}

```

Örnek: {1, 3, 12, 21, 43, 57, 61, 82}, **target=57**, $j = \lfloor \sqrt{8} \rfloor = 2$



İterasyon Analizi

Blok atlama: $\lfloor n/j \rfloor$ adım | Blok içi linear: $j - 1$ adım

$$T(n, j) = \left\lfloor \frac{n}{j} \right\rfloor + (j - 1)$$

Optimal j bulma: $\frac{dT}{dj} = 0$

$$\frac{d}{dj} \left(\frac{n}{j} + j \right) = -\frac{n}{j^2} + 1 = 0 \implies j^2 = n \implies j = \sqrt{n}$$

$$T(n) = \frac{n}{\sqrt{n}} + \sqrt{n} - 1 = \sqrt{n} + \sqrt{n} - 1 = 2\sqrt{n} - 1$$

Θ ispatı: $f(n) = \sqrt{n}$

$$c_1 \sqrt{n} \leq 2\sqrt{n} - 1 \leq c_2 \sqrt{n} \implies c_1 \leq 2 - \frac{1}{\sqrt{n}} \leq c_2$$

$n_0 = 1, c_1 = c_2 = 1$:

$$T(n) = \Theta(\sqrt{n})$$

Pratik – Jump Search

- (a) $n = 100$ için optimal blok boyutu nedir? Toplam kaç adım atar? _____
- (b) Binary Search'e göre Jump Search'ün avantajı nedir? ($O(\log n)$ vs $O(\sqrt{n})$) _____

4. Interpolation / Dictionary Search

“Sözlükte K kelimesini ararken sayfanın ortasına değil, K harfine yakın bir yere açılır.” Uniform dağılımlı verilerde Binary Search'ten hızlıdır.

Interpolation / Dictionary Search – Tahminle Arama

Fikir: Doğrusal Enterpolasyon ile Tahmin

Binary Search: her seferinde $\text{mid} = \lfloor (\text{left} + \text{right})/2 \rfloor$ kullanır.

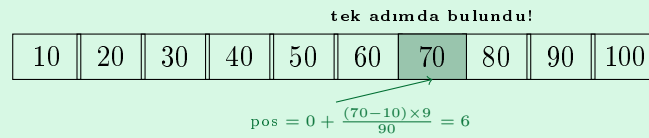
Interpolation Search: elemanın yerini *tahmin* eder:

$$\text{pos} = \text{left} + \left\lfloor \frac{(\text{target} - \text{arr}[\text{left}]) \cdot (\text{right} - \text{left})}{\text{arr}[\text{right}] - \text{arr}[\text{left}]} \right\rfloor$$

Kod

```
int interpolationSearch(int arr[], int n, int target) {
    int left = 0, right = n-1;
    while (left <= right
        && target >= arr[left] && target <= arr[right]) {
        if (left == right) {
            if (arr[left] == target) return left;
            return -1;
        }
        int pos = left + (long)(target - arr[left])
            * (right - left)
            / (arr[right] - arr[left]);
        if (arr[pos] == target) return pos;
        if (arr[pos] < target) left = pos + 1;
        else right = pos - 1;
    }
    return -1;
}
```

Örnek: Uniform dağılım



Karmaşıklık Analizi

Durum	Veri Dağılımı	Karmaşıklık
En iyi	Tek adımda bulunur	$O(1)$
Ortalama	Uniform (eşit aralıklı)	$O(\log \log n)$
En kötü	Uzak dağılım (exponential vb.)	$O(n)$

$O(\log \log n)$ **sezgisi:** Binary Search her adımda $n \rightarrow n/2$ yapar (logaritmik büyüme). Interpolation Search her adımda arama uzayını $n \rightarrow \sqrt{n}$ yapar (büyüme hızı karesi alınır):

$$n \rightarrow n^{1/2} \rightarrow n^{1/4} \rightarrow \dots \rightarrow 1 \implies k = \log \log n \text{ adım}$$

$$T(n) = O(\log \log n) \text{ (uniform dağılım)}$$

$$T(n) = O(n) \text{ (en kötü durum)}$$

Dictionary Search $\equiv$ Interpolation Search

“Dictionary Search” ve “Interpolation Search” aynı algoritmadır. Adı sözlük analojisinden gelir: “A” harfini ararken sayfanın başını açılır, “Z” için sonunu. Enterpolasyon formülü tam olarak bu tahmini yapar.

Pratik – Interpolation Search

(a) $\{2, 4, 6, 8, 10, 12, 14, 16\}$, target=10 için pos formülüyle ilk tahmini hesaplayın. _____

(b) Neden $\{1, 2, 4, 8, 16, 32\}$ (geometrik dizi) Interpolation Search için en kötü durum oluşturur? _____

5. Quadratic Binary Search

“Exponential Search” olarak da bilinir. Önce $1, 2, 4, 8, \dots$ ile hedefin bulunduğu bloğu bulur, sonra o blokta Binary Search yapar. Sınırı bilinmeyen veya çok büyük dizilerde iyidir.

Quadratic Binary Search – Karesel İkili Arama

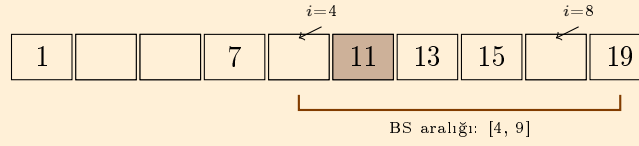
Fikir

1. $i = 1$ 'den başla, $arr[i] < \text{target}$ olduğu sürece $i \leftarrow 2i$ (ikiye katla).
2. $arr[i] \geq \text{target}$ olduğunda: $[i/2, \min(i, n-1)]$ aralığında Binary Search.

Kod

```
int exponentialSearch(int arr[], int n, int target) {
    if (arr[0] == target) return 0;
    int i = 1;
    while (i < n && arr[i] <= target)
        i *= 2; // ikiye katla
    // Binary Search: [i/2, min(i, n-1)]
    return binarySearch(arr, i/2, min(i, n-1), target);
}
```

Görsel



Karmaşıklık Analizi

1. **Aşama (ikiye katlama):** $arr[i] \leq \text{target}$ olduğu sürece devam eder. Target dizinin k . indeksindeyse, $2^{\lfloor \log k \rfloor}$ adımda blok bulunur.

2. **Aşama (Binary Search):** $[i/2, i]$ aralığı, yani $i/2$ boyutlu alt dizi: $T_{BS} = O(\log(i/2)) = O(\log k)$

$$T(n) = O(\log k) + O(\log k) = O(\log k) = O(\log n)$$

$$T(n) = O(\log n)$$

Üst sınır: $2 \cdot O(\log n) = O(\log n)$ (sabit katsayı yok olur)

Durum	Aşama 1	Aşama 2	Toplam
En iyi ($arr[0]$)	$O(1)$	$O(1)$	$O(1)$
Ortalama / En kötü	$O(\log n)$	$O(\log n)$	$O(\log n)$

Ne zaman kullanılır? Dizi boyutu bilinmiyorsa veya çok büyükse (akış, sınırsız dizi). Binary Search'e göre avantajı: başlangıçta $O(n)$ yerine $O(\log n)$ 'de doğru aralığı bulur.

Pratik – Quadratic (Exponential) Binary Search

(a) $\{3, 6, 9, 12, 15, 18, 21, 24, 27, 30\}$, $\text{target}=21$ için her ikiye katlama adımını yazın. Kaç. adımda Binary Search başlar? _____

(b) Neden dizinin başında aranan elemanlar için Binary Search'ten daha hızlıdır? ____

6. Özet Tablosu

Algoritma	Koşul	En İyi	Ortalama	En Kötü	Alan
Linear Search	Sırasız	$O(1)$	$O(n)$	$O(n)$	$O(1)$
Binary Search	Sıralı	$O(1)$	$O(\log n)$	$O(\log n)$	$O(1)$
Jump Search	Sıralı	$O(1)$	$O(\sqrt{n})$	$O(\sqrt{n})$	$O(1)$
Interpolation	Sıralı+ Uni-form	$O(1)$	$O(\log \log n)$	$O(n)$	$O(1)$
Exp./Quadratic BS	Sıralı	$O(1)$	$O(\log n)$	$O(\log n)$	$O(1)$

Seçim Rehberi:

- Veri sırasız $\rightarrow$ **Linear Search**
- Veri sıralı, boyut biliniyor $\rightarrow$ **Binary Search**
- Veri sıralı, uniform dağılım $\rightarrow$ **Interpolation Search** ($O(\log \log n)$)
- Veri sıralı, boyut bilinmiyor/sonsuz $\rightarrow$ **Exponential/Quadratic BS**
- Bloklü bellekte (disk, tape) $\rightarrow$ **Jump Search**